

Rapport sur le projet ThanosPizza

Table des matières

Introduction.....	2
Rappel sur la configuration de docker	2
Chargement des données	2
Sauvegarde de la base de données	4
Backup complet	4
Backup différentielle	5
Requêtes	5
Requête 1.....	6
Requête 2.....	6
Requête 3.....	6
Requête 4.....	7
Requête 5.....	7
Requête 6.....	7
Requête 7.....	7
Requête 8.....	8
Requête 9.....	8
Requête 10	8
Index	9
Index 1	9
Index 2	9
Utilisateurs et rôles	10
1. Créer les rôles	10
2. Attribuer les permissions aux rôles	10
3. Vérifier les privilèges des rôles	11
4. Créer les utilisateurs	13
5. Attribuer les rôles aux utilisateurs	14
Conclusion	14
Générale	14

Personnelle	14
-------------------	----

Introduction

Au cours de ce projet, je serais chargé de créer et gérer une base de données regroupant des informations à propos des commandes d'une pizzeria.

Nous pouvons découper ce projet en 5 tâches :

1. Créer les modèles de données MLD – MCD à l'aide de looping, modéliser la base de données et la faire valider au client.
2. Insérer des données fournies par le client dans la base de données à l'aide d'un LOAD DATA simple.
3. Sauvegarder la base de données (régulièrement) afin de ne rien perdre en cas de catastrophe. Le rapport technique expliquera la méthode de sauvegarde.
4. Sélectionner des données / statistiques pour le client à l'aide de requêtes SQL. Par la même occasion, nous validerons l'intégrité des données.
5. Rédiger un rapport technique (celui-ci) décrivant les 4 étapes ci-dessus.

Rappel sur la configuration de docker

Le docker-compose est trouvable dans ...\\106-P_DB-ThanosPizza\\Docker_MySQL

Il a été modifié afin de pouvoir lier le dossier scripts et mysql. Pour ceci la partie volumes du fichier a été légèrement modifiée. Voici ce que ça donne après la modification.

volumes:

- dbdata:/var/lib/mysql
- ./scripts:/var/lib/mysql-files

Chargement des données

Il m'a été donné une panoplie de fichiers.tsv, qui sont l'équivalent de fichiers .csv mais utilisant des tabulations à la place des virgules afin de séparer les données. Ces fichiers contiennent toutes les données à insérer et voici ce que j'ai fait pour les loader.

Premier pas : créer la structure de la base de données. Facile il suffit de copier-coller le script LDD de Looping, après avoir ajouté l'énoncé CREATE-USE database manuellement, vers un fichier texte. Il faut ensuite placer le fichier texte dans le répertoire scripts de docker et l'importer à l'aide de la commande suivante à jouer dans la console "mysql -u root -proot < /var/lib/mysql-files/script_creation.sql".

Nous voilà à l'étape du chargement des données

-- table 1

```
LOAD DATA INFILE '/var/lib/mysql-files/t_article.tsv'
  INTO TABLE t_produit
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES;
```

-- table 2

```
LOAD DATA INFILE '/var/lib/mysql-files/t_livreur.tsv'
  INTO TABLE t_livreur
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES;
```

-- table 3

```
LOAD DATA INFILE '/var/lib/mysql-files/t_client.tsv'
  INTO TABLE t_client
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES;
```

-- table 4

```
LOAD DATA INFILE '/var/lib/mysql-files/t_adresse.tsv'
  INTO TABLE t_adresse
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES
  (`adresse_id`, `client_fk`, `rue`, `npa`, `localite`, `latitude`, `longitude`);
```

-- table 5

```
LOAD DATA INFILE '/var/lib/mysql-files/t_commande.tsv'
  INTO TABLE t_commande
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES
  (commande_id, client_fk, type, @vAdresse, @mytimestamp, statut)
  SET adresse_fk = NULLIF(@vAdresse,''),
  date_et_heure = STR_TO_DATE(@mytimestamp, '%d.%m.%Y %H:%i');
```

-- table 6

```
LOAD DATA INFILE '/var/lib/mysql-files/t_ligne_commande.tsv'
  INTO TABLE t_ligne_de_commande
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES
  (` ligne_de_commande_id`, ` commande_fk`, ` produit_fk`, ` quantite`,
   ` prix_unitaire`, @extra)
  SET produit_enfant_fk = NULLIF(@extra, "");
```

-- table 7

```
LOAD DATA INFILE '/var/lib/mysql-files/t_paiement.tsv'
  INTO TABLE t_paiement
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES
  (` paiement_id`, ` commande_fk`, ` mode`, ` montant`, @mytimestamp)
  SET date_paiement =STR_TO_DATE(@mytimestamp, '%d.%m.%Y %H:%i');
```

-- table 8

```
LOAD DATA INFILE '/var/lib/mysql-files/t_livraison.tsv'
  INTO TABLE t_livraison
  CHARACTER SET UTF8
  COLUMNS TERMINATED BY '\t'
  IGNORE 1 LINES
  (` livraison_id`, ` commande_fk`, ` livreur_fk`, ` statut`, @depart, @arrivee)
  SET heure_depart =STR_TO_DATE(@depart, '%d.%m.%Y %H:%i')
  heure_arrivee =STR_TO_DATE(@arrivee, '%d.%m.%Y %H:%i');
```

Sauvegarde de la base de données

Dans ce chapitre, nous allons nous intéresser à la manière dont nous allons sauvegarder notre base de données. Il est prévu que nous fassions une sauvegarde complète toutes les semaines et une sauvegarde différentielle chaque jour.

Backup complet

Nous allons commencer par la sauvegarde complète une fois par semaine. Voici la commande à lancer une fois par semaine.

```
mysqldump -u root -proot --add-drop-database --lock-all-tables db_Thanoss_Pizza >
/var/lib/mysql-files/fullBackup-db_Thanoss_Pizza-$(date +%d-%m-%Y).sql
```

Backup différentielle

Ici, afin de réduire la quantité d'énergie et de temps consommé par la sauvegarde, nous allons demander à mysql de ne sauvegarder que les différences avec la dernière sauvegarde complète. Avant de créer le backup, nous devons faire quelques modifications sur toutes les tables de la base de données. On modifie donc la structure des tables afin que lorsque l'on modifie une valeur, la date de modification se met à jour. Ainsi on peut sauvegarder que les données qui ont été changées.

```
-- t_client
ALTER TABLE t_client ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_paiement
ALTER TABLE t_paiement ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_adresse
ALTER TABLE t_adresse ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_commande
ALTER TABLE t_commande ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_ligne_de_commande
ALTER TABLE t_ligne_de_commande ADD COLUMN derniere_modification TIMESTAMP
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_produit
ALTER TABLE t_produit ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_livraison
ALTER TABLE t_livraison ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
-- t_livreur
ALTER TABLE t_livreur ADD COLUMN derniere_modification TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
```

Et voici la commande à lancer chaque jour.

```
mysqldump -u root -proot db_Thanoss_Pizza --no-create-db --no-create-info --
where="derniere_modification > '2025-10-10 00:00:00'" > /var/lib/mysql-
files/differentialBackup-db_Thanoss_Pizza-$(date +%d-%m-%Y).sql
```

Restauration

Afin de restaurer la database, nous allons tout simplement restaurer la sauvegarde complète, puis la dernière sauvegarde différentielle si nécessaire. Voici le procédé.

```
-- sauvegarde complète
```

```
mysql -u root -proot db_Thanoss_Pizza < /var/lib/mysql-files/ fullBackup-  
db_Thanoss_Pizza-[la date].sql  
-- backup différentielle  
mysql -u root -proot db_Thanoss_Pizza < /var/lib/mysql-files/differentialBackup-  
db_Thanoss_Pizza-[la date].sql
```

Requêtes

Dans ce chapitre, nous allons explorer diverses possibilités avec les requêtes SELECT

Requête 1

Afficher les dix pizzas les plus vendues (sans les toppings), triés par quantités totales décroissantes. Vous devez afficher le nom et les quantités.

```
select nom, count(produit_fk) as quantite_vendue from t_ligne_de_commande  
  join t_produit on produit_id = produit_fk  
 where type LIKE 'pizza'  
 group by nom  
 order by quantite_vendue desc  
 limit 10;
```

Requête 2

Afficher les toppings les plus ajoutés. Le résultat doit être ordonné par le nombre de toppings de manière décroissante. Vous devez afficher le nom et le nombre.

```
select nom, count(produit_enfant_fk) as nombre from t_ligne_de_commande  
  join t_produit on produit_id = produit_fk  
 where produit_enfant_fk is not null  
 group by nom  
 order by nombre desc ;
```

Requête 3

Afficher le chiffre d'affaires par jour (commandes livrées). Vous ne devez afficher que la date et le chiffres d'affaires (arrondi à 2 chiffres après la virgule).

```
select (select date (date_paiement)) as date, round(sum(montant), 2) from t_paiement  
  join t_commande on commande_id = commande_fk  
 where statut LIKE 'livr_e_'  
 group by date;
```

Requête 4

Afficher le chiffre d'affaires par NPA (adresse de livraison).

1ère colonne : npa

2ème colonne : localité

3ème colonne : chiffre d'affaires (arrondi à 2 chiffres après la virgule)

```
select npa, localite, round(sum(montant), 2) as chiffre_affaire from t_adresse
      join t_commande on adresse_id = adresse_fk
      join t_paiement on commande_id = commande_fk
group by npa, localite;
```

Requête 5

Affiche le nombre de commandes par heure. Il s'agit par cette requêtes de savoir quelles sont les heures « chaudes ».

NB : les heures « chaudes » sont des heures pendant lesquelles le nombre de commandes sont les plus élevées.

```
select count(commande_id) as nombre_commandes, date_format(date_et_heure,
"%H") as heure from t_commande
      group by heure
      order by nombre_commandes desc;
```

Requête 6

Afficher le nombre de commandes des clients les plus fidèles. Un client est fidèle si son nombre de commandes est ≥ 5 . Afficher le résultat par ordre décroissant du nombre de commandes, puis par ordre alphabétique du nom.

```
select count(client_fk) as nombre_commandes, nom, prenom from t_commande
      join t_client on client_id = client_fk
      group by nom, prenom
      having count(client_fk) >= 5
      order by nombre_commandes desc, nom;
```

Requête 7

Afficher le total dû par commande. Afficher l'id de la commande et le montant dû (arrondi à 2 chiffres après la virgule). Ordonnez le résultat par ordre croissant des ids de commandes.

```
select round(sum(prix_unitaire * quantite), 2) as montant, commande_id from
t_ligne_de_commande
  join t_commande on commande_id = commande_fk
  group by commande_id
  order by commande_id;
```

Requête 8

Afficher le total payé par commande (commande ayant au moins un paiement). Afficher l'id de la commande et le total payé (arrondi à 2 chiffres après la virgule). Ordonnez le résultat par ordre croissant des ids de commandes.

```
select round(sum(montant), 2), commande_id from t_paiement
  join t_commande on commande_id = commande_fk
  group by commande_id
  order by commande_id;
```

Requête 9

Quelle est la répartition des types de commandes.

Ordonner le résultat par le nombre de commande de chaque type, du plus grand au plus petit.

1ère colonne : type

2ème colonne : nombre de commandes de ce type

```
select type, count(commande_id) as nombre_commandes from t_commande
  group by type
  order by nombre_commandes desc;
```

Requête 10

Quel est le délai moyen de livraison par livreur (en minutes).

Ordonner le résultat par délai moyen en minutes du plus petit au plus grand.

Aide : l'id du livreur, son nom et le délai dans le SELECT.

```
select nom, round(avg(heure_arrivee - heure_depart)/100, 2) as delai_moyen from
t_livraison
  join t_livreur on livreur_id = livreur_fk
  group by nom
  order by delai_moyen asc;
```


Index

Dans ce chapitre, nous allons créer des index afin d'optimiser la recherche de données en temps et en énergie. L'insertion en revanche prendra plus de temps, mais ce ne sera pas perceptible avant longtemps.

Index 1

```
SELECT c.commande_id, c.date_et_heure, c.statut, cl.nom AS client
FROM t_commande AS c
JOIN t_client AS cl ON c.client_fk = cl.client_id
WHERE c.statut = 'en_livraison' AND c.date_et_heure > '2025-01-01'
ORDER BY c.date_et_heure DESC;
```

Voici la commande de création de l'index le plus optimisé pour cette commande.

```
create index idx_id_date_statut on t_commande(commande_id, date_et_heure, statut);
```

Index 2

```
SELECT a.npa AS zone_npa, COUNT(c.commande_id) AS nb
FROM t_commande AS c
JOIN t_adresse AS a ON c.adresse_fk = a.adresse_id
WHERE c.type = 'livraison' AND HOUR(c.date_et_heure) BETWEEN 18 AND 21
GROUP BY a.npa
ORDER BY nb DESC;
```

À présent, nous nous attaquons au 2e index. Voici la commande.

```
create index idx_id_type_date on t_commande(commande_id, type, date);
```

Utilisateurs et rôles

Nous allons maintenant créer de nouveaux utilisateurs et leur donner des permissions à l'aide de rôles. Nous utiliserons des rôles afin de pouvoir assigner les permissions aux autres utilisateurs plus rapidement. Nous allons procéder par étapes.

1. Créer les rôles.
2. Donner des permissions aux rôles.
3. Vérifier les privilèges des rôles.
4. Créer les utilisateurs.
5. Assigner les rôles aux utilisateurs

1. Créer les rôles

Nous commençons par créer tous les rôles “vides”, nous leur donnerons des permissions plus tard.

```
create role 'Administrateur', 'Manager', 'Pizzaiolo', 'Livreur', 'Agent de caisse', 'Analyste';
```

2. Attribuer les permissions aux rôles

Nos rôles sont maintenant actifs mais sans permissions, nous allons donc leur en donner. Vous pouvez copier-coller les lignes ci-dessous.

```
-- administrateur
grant all privileges on db_Thanoss_Pizza.* to 'Administrateur';
grant grant option on db_Thanoss_Pizza.* to 'Administrateur';
-- manager
grant update on db_Thanoss_Pizza.t_commande to 'Manager';
grant update on db_Thanoss_Pizza.t_livraison to 'Manager';
grant select on db_Thanoss_Pizza.t_paiement to 'Manager';
grant update on db_Thanoss_Pizza.t_produit to 'Manager';
-- pizzaiolo
grant select on db_Thanoss_Pizza.t_commande to 'Pizzaiolo';
grant select on db_Thanoss_Pizza.t_commande to 'Pizzaiolo';
grant update(statut) on db_Thanoss_Pizza.t_commande to 'Pizzaiolo';
-- livreur
grant select on db_Thanoss_Pizza.t_livraison to 'Livreur';
grant select on db_Thanoss_Pizza.t_commande to 'Livreur';
grant update(statut) on db_Thanoss_Pizza.t_livraison to 'Livreur';
-- agent de caisse
grant select on db_Thanoss_Pizza.t_paiement to 'Agent de caisse';
grant insert on db_Thanoss_Pizza.t_paiement to 'Agent de caisse';
```

```
grant select on db_Thanoss_Pizza.t_commande to 'Agent de caisse';  
-- analyste  
grant select on db_Thanoss_Pizza.* to 'Analyste';
```

3. Vérifier les privilèges des rôles

Avant d'attribuer les rôles aux utilisateurs, il est vital de vérifier les privilèges de chaque rôle. Ainsi nous allons utiliser cette commande :

```
SHOW GRANTS FOR 'role';
```

Où nous remplacerons 'role' par le véritable nom du rôle. Voici les commandes à entrer et en screenshot, les résultats attendus.

```
-- administrateur
```

```
SHOW GRANTS FOR 'Administrateur';
```

```
-- manager
```

```
SHOW GRANTS FOR 'Manager';
```

```
-- pizzaiolo
```

```
SHOW GRANTS FOR 'Pizzaiolo';
```

```
-- livreur
```

```
SHOW GRANTS FOR 'Livreur';
```

```
-- agent de caisse
```

```
SHOW GRANTS FOR 'Agent de caisse';
```

```
-- analyste
```

```
SHOW GRANTS FOR 'Analyste';
```

```

mysql> -- administrateur
mysql> SHOW GRANTS FOR 'Administrateur';
+-----+
| Grants for Administrateur@% |
+-----+
| GRANT USAGE ON *.* TO `Administrateur`@`%` |
| GRANT ALL PRIVILEGES ON `db_Thanoss_Pizza`.* TO `Administrateur`@`%` WITH GRANT OPTION |
+-----+
2 rows in set (0.00 sec)

mysql> -- manager
mysql> SHOW GRANTS FOR 'Manager';
+-----+
| Grants for Manager@% |
+-----+
| GRANT USAGE ON *.* TO `Manager`@`%` |
| GRANT UPDATE ON `db_Thanoss_Pizza`.`t_commande` TO `Manager`@`%` |
| GRANT UPDATE ON `db_Thanoss_Pizza`.`t_livraison` TO `Manager`@`%` |
| GRANT SELECT ON `db_Thanoss_Pizza`.`t_paielement` TO `Manager`@`%` |
| GRANT UPDATE ON `db_Thanoss_Pizza`.`t_produit` TO `Manager`@`%` |
+-----+
5 rows in set (0.00 sec)

mysql> -- pizzaiolo
mysql> SHOW GRANTS FOR 'Pizzaiolo';
+-----+
| Grants for Pizzaiolo@% |
+-----+
| GRANT USAGE ON *.* TO `Pizzaiolo`@`%` |
| GRANT SELECT, UPDATE (`statut`) ON `db_Thanoss_Pizza`.`t_commande` TO `Pizzaiolo`@`%` |
+-----+
2 rows in set (0.00 sec)

mysql> -- livreur
mysql> SHOW GRANTS FOR 'Livreur';
+-----+
| Grants for Livreur@% |
+-----+
| GRANT USAGE ON *.* TO `Livreur`@`%` |
| GRANT SELECT ON `db_Thanoss_Pizza`.`t_commande` TO `Livreur`@`%` |
| GRANT SELECT, UPDATE (`statut`) ON `db_Thanoss_Pizza`.`t_livraison` TO `Livreur`@`%` |
+-----+
3 rows in set (0.00 sec)

mysql> -- agent de caisse
mysql> SHOW GRANTS FOR 'Agent de caisse';
+-----+
| Grants for Agent de caisse@% |
+-----+
| GRANT USAGE ON *.* TO `Agent de caisse`@`%` |
| GRANT SELECT ON `db_Thanoss_Pizza`.`t_commande` TO `Agent de caisse`@`%` |
| GRANT SELECT, INSERT ON `db_Thanoss_Pizza`.`t_paielement` TO `Agent de caisse`@`%` |
+-----+
3 rows in set (0.00 sec)

mysql> -- analyste
mysql> SHOW GRANTS FOR 'Analyste';
+-----+
| Grants for Analyste@% |
+-----+
| GRANT USAGE ON *.* TO `Analyste`@`%` |
| GRANT SELECT ON `db_Thanoss_Pizza`.* TO `Analyste`@`%` |
+-----+

```

4. Créer les utilisateurs

À présent, nous avons les rôles à jour. Nous allons donc créer des utilisateurs, n'ayant pas de spécifications pour les noms et mot de passe de ces derniers, nous allons les définir ici. Tous les mots de passe des utilisateurs correspondent à leur nom. Voici la commande à entrer afin de créer les users.

```
create user
'Thanoss_Pizza admin'@'localhost' identified by 'Thanoss_Pizza admin',
'Thanoss_Pizza manager'@'localhost' identified by 'Thanoss_Pizza manager',
'Thanoss_Pizza pizzaiolo'@'localhost' identified by 'Thanoss_Pizza pizzaiolo',
'Thanoss_Pizza livreur'@'localhost' identified by 'Thanoss_Pizza livreur',
'Thanoss_Pizza agent de caisse'@'localhost' identified by 'Thanoss_Pizza agent de
caisse',
'Thanoss_Pizza analyste'@'localhost' identified by 'Thanoss_Pizza analyste';
```

Vérifions que la creation s'est bien déroulée :

```
-- la commande
use mysql;
select User, host from user;
```

Thanoss_Pizza admin	localhost
Thanoss_Pizza agent de caisse	localhost
Thanoss_Pizza analyste	localhost
Thanoss_Pizza livreur	localhost
Thanoss_Pizza manager	localhost
Thanoss_Pizza pizzaiolo	localhost

5. Attribuer les rôles aux utilisateurs

Finalement, nous allons donner un rôle à chaque utilisateur, voici les commandes :

```
-- administrateur
grant 'Administrateur' to 'Thanoss_Pizza admin'@'localhost';
SET DEFAULT ROLE 'Administrateur' TO 'admin'@'localhost';
-- manager
grant 'Manager' to 'Thanoss_Pizza manager'@'localhost';
SET DEFAULT ROLE 'Manager' TO 'Thanoss_Pizza manager'@'localhost';
-- pizzaiolo
grant 'Pizzaiolo' to 'Thanoss_Pizza pizzaiolo'@'localhost';
SET DEFAULT ROLE 'Pizzaiolo' TO 'Thanoss_Pizza pizzaiolo'@'localhost';
-- livreur
grant 'Livreur' to 'Thanoss_Pizza livreur'@'localhost';
SET DEFAULT ROLE 'Livreur' TO 'Thanoss_Pizza livreur'@'localhost';
-- agent de caisse
grant 'Agent de caisse' to 'Thanoss_Pizza agent de caisse'@'localhost';
SET DEFAULT ROLE 'Agent de caisse' TO 'Thanoss_Pizza agent de caisse'@'localhost';
-- analyste
grant 'Analyste' to 'Thanoss_Pizza analyste'@'localhost';
SET DEFAULT ROLE 'Analyste' TO 'Thanoss_Pizza analyste'@'localhost';
```

Conclusion

Générale

Personnelle